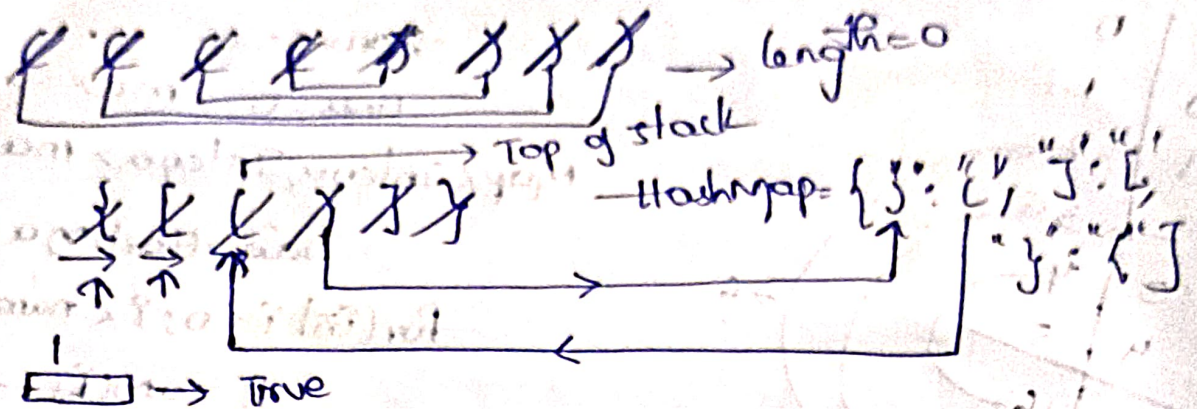


20. valid parentheses

s = "([)]" → True

s = "([)]" → False
different

Ex:

Time - $O(n)$ → 1-passSpace - $O(n)$ → stackPython

```
d = { "(": ")", "[": "]", "{": "}" }
stack = []
```

```
for s in str:
    if s not in hash:
        stack.append(s)
```

```
else:
    if stack:
        e = stack.pop()
        if (e != d[s]):
            return false
```

```
if (len(stack) == 0):
    return True
```

```
else:
    return false
```

```
stack = new stack<>()
map<char, char> map = new HashMap<>()
map.put("(", ")")
```

```
for (int i = 0; i < s.length(); i++) {
    char ch = s.charAt(i);
    if (map.containsKey(ch)) {
        if (!stack.isEmpty() && stack.peek().equals(map.get(ch)))
            stack.pop();
        else return false;
    } else stack.push(ch);
}
```

```
if (stack.isEmpty()) return true;
else return false;
```

155.

Min Stack

(2)

→ Design a stack that supports push, pop, top and getMin in O(1) time.

[MinStack, push, push, push, getMin, pop, top, getMin]

[[], [-2], [0], [-3], [], [], [], []]

→ stack support O(1) for push, pop & top but getMin() - ?

Bruteforce → traversing the stack again to find min - O(n)
caching value in variable → But if element pop(), what next minimum?

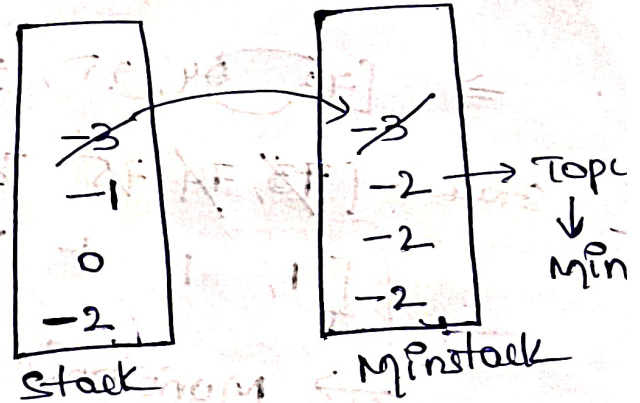
→ so, solution → maintain another stack (minStack), which stores min value of every element correspondingly

```

JAVA class MinStack {
    private Stack<Integer> stack;
    private Stack<Integer> minStack;

    const ← public MinStack() {
        stack = new Stack<>();
        minStack = new Stack<>();
    }

```



main logic

```

    public void push(int val) {
        stack.push(val);
        val = Math.min(val, (minStack.isEmpty() ? val : minStack.peek()));
        minStack.push(val);
    }

```

```

    public void pop() {
        stack.pop();
        minStack.pop();
    }

```

```

public int top() {
    return stack.peek()
}

```

```

public int getMin() {
    return minstack.peek()
}

```

* 739. Daily Temperatures

[73, 74, 75, 71, 69, 72, 76, 73]

op $\Rightarrow$ [1, 1, 4, 2, 1, 1, 0, 0]

$\Rightarrow$ [73, 74, 75, 71, 69, 72, 76, 73]

stack $\rightarrow$ [~~73~~, ~~74~~, ~~75~~, ~~71~~, ~~69~~, ~~72~~, 76, 73]

[1, 1, 4, 2, 1, 1, 0, 0]

$\rightarrow$ Monotonic decreasing stack

[30, 40, 50, 60] | [8, 3, 1, 4, 9, 7, 2, 6, 3]

stack = [~~30~~, ~~40~~, ~~50~~, 60] | [~~8~~, ~~3~~, ~~1~~, 4, 9, 7, ~~2~~, 6, 3]

res = [1, 1, 1, 0] | res = [4, 2, 1, 1, 0, 0, 1, 0, 0]

monotonic decrease stack will always decrease

Python

res = [] * 0

stack = []

stack[0] = arr[0]

for i, n in enumerate(arr):

if (stack.peek()

res = [] * len(temp)

stack = [] $\Rightarrow$ # pair [temp index]

for i, t in enumerate(temp):

while (stack and t > stack[-1][0]):

stack[-1][1] = stack.pop()

res[stack[-1][1]] = t - stack[-1][1]

stack.append([t, i])

return res

* JAVA

public int[] dailyTemperature(int[] temp) {

int[] res = new int[temp.length];

Arrays.fill(res, 0);

Stack<int> stack = new Stack<>();

for (int i = 0; i < temp.length; i++) {

while (!stack.isEmpty() && temp[i] > stack.peek()[0]) {

int[] popElement = stack.pop();

int stackIndex = popElement[1];

res[stackIndex] = i - stackIndex;

}

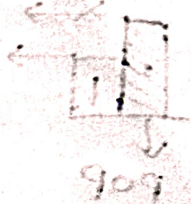
stack.push(new int[] {temp[i], i});

}

return res;

}

8, 3, 1, 4, 9, 2, 6, 3
~~stack~~ ~~[[8, 0], [3, 1], [1, 2], [4, 3], [9, 4], [2, 5], [6, 6], [3, 7]]~~
~~res~~ ~~[[4, 1], [2, 2], [1, 3], [9, 4], [2, 5], [6, 6], [3, 7]]~~

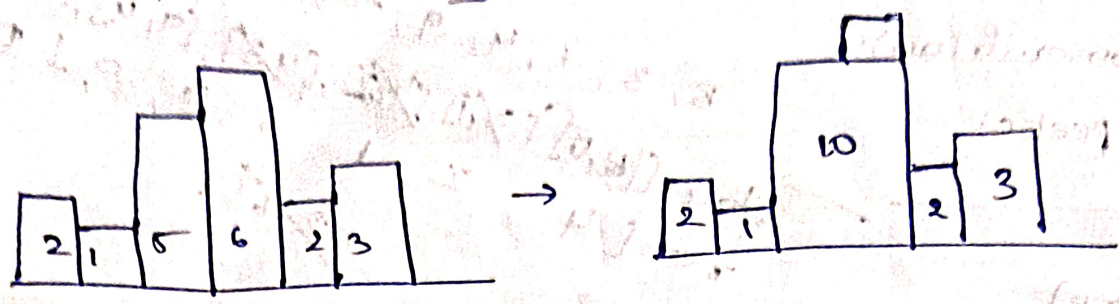


destructing
the pop()

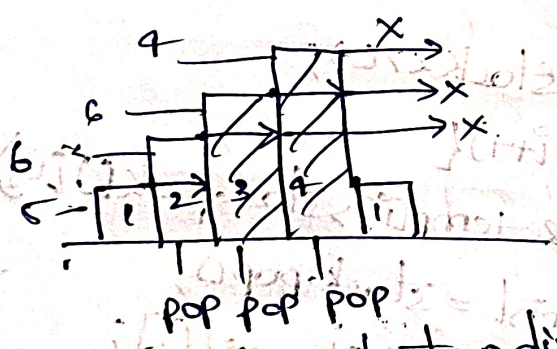
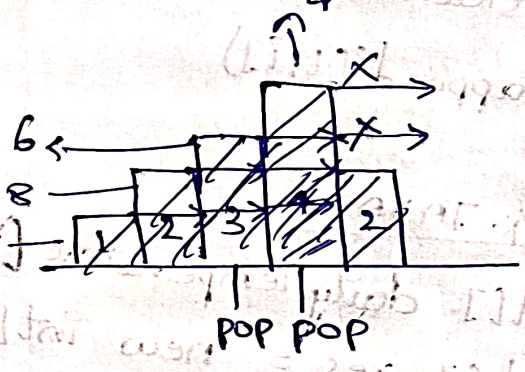
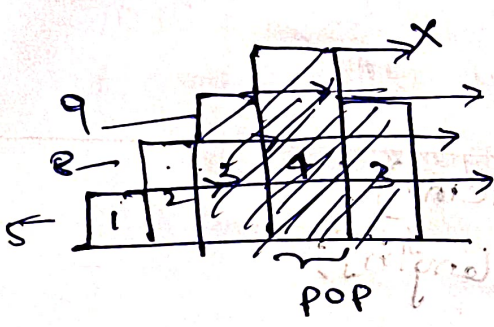
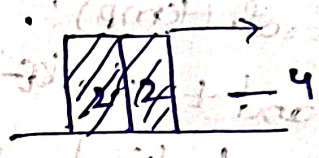
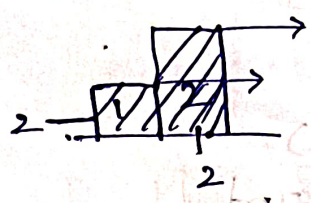
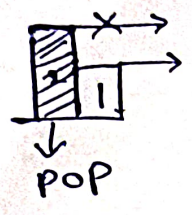
Q8

84. Largest Rectangle in Histogram $\rightarrow$ monotonic increasing stack

$[2, 1, 5, 6, 2, 3] \rightarrow 10$



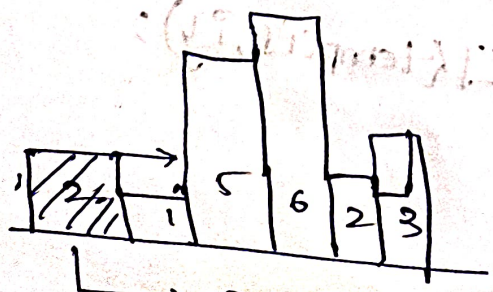
diagrammatic analysis



$\rightarrow$ we use pop from top to bottom so using stack

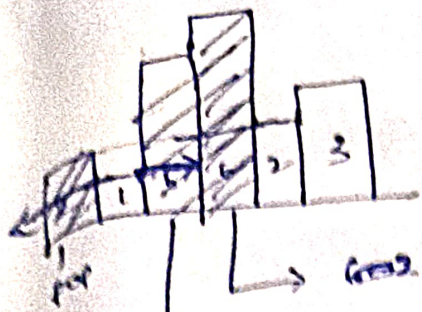
conceptual understanding

stack



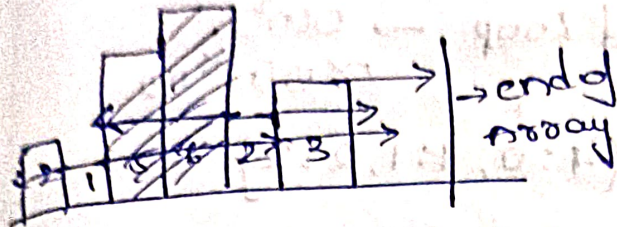
$\text{Pop} \rightarrow (1-0) * 2 = 2$

index	value	max
0	2	$\downarrow$ 2
1	1	$\rightarrow$ pop



$$(4-3) \times 4 = 4$$

$$(4-2) \times 5 = 10$$



max $\rightarrow$ 10

⑥

stack index	value	max
0	1	
1	5	
2	4	
3	3	10

$6 > 2 \rightarrow \text{POP}$

stack index	value	
0	1	$(6-0) \times 1 = 6$
1	2	$(6-4) \times 2 = 4$
2	3	$(6-5) \times 3 = 3$

max = 10

JAVA

```
public int largestRectangle(int[] heights) {
```

```
    stack<int[]> stack = new stack<>();
```

```
    int max = 0;
```

```
    for(int i = 0; i < heights.length; i++) {
```

```
        int popstack = i;
```

```
        while(!stack.isEmpty() && stack.peek()[0] > heights[i])
```

```
            int area = (i - stack.peek()[0]) * stack.peek()[1];
```

```
            max = max(area, max);
```

```
            stack.pop();
```

```
            popstack = stack.peek()[0];
```

```
        stack.push(new int[] { popstack, heights[i] });
```

```
    } while(!stack.isEmpty()) {
```

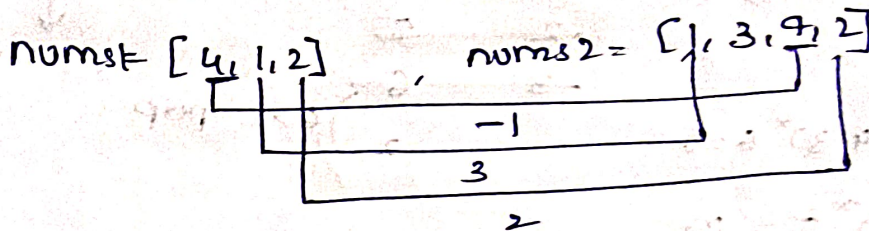
```
        int area = (heights.length - stack.peek()[0]) * stack.peek()[1];
```

```
        max = max(max, area);
```

```
        stack.pop();
```

```
    }
```

796. Next greater Element I



⇒ [-1, 3, 2]

Approach 1 → Hash Map + Nested Loop → $O(n^2)$

Map = {4:0, 1:1, 2:2}

Check map: [4, 1, 2] → Map

res = [-1, -1, -1] = [-1, 3, -1]

JAVA:

```

public int[] nextGreater(int[] nums1, int[] nums2) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums1.length; i++) {
        map.put(nums1[i], i);
    }
    int res[] = new int[nums1.length];
    Arrays.fill(res, -1);
    for (int i = 0; i < nums2.length; i++) {
        if (map.containsKey(nums2[i])) {
            for (int j = i + 1; j < nums2.length; j++) {
                if (nums2[j] > nums2[i]) {
                    res[map.get(nums2[i])] = nums2[j];
                    break;
                }
            }
        }
    }
    return res;
}
    
```

Approach 2

[4, 1, 2]

⇒ [2, 1, 3, 4]

→ Monotonical
increasing stack
decreasing

→ 6, 1, 3, 4, 5, 7

[6, 1, 3, 4, 5, 7] → greater

public class Int[] nextGreatest (int[] nums1, int[] nums2) {

Stack<Integer> stack = new Stack<>();

Map<Integer, Integer> map = new HashMap<>();

Quick
retroval

int[] res = new int[nums2.length];

for (int i = 0; i < nums2.length; i++) {

while (!stack.isEmpty() && stack.peek() < nums2[i]) {

res[map.get(stack.peek)] = nums2[i];

stack.pop();

if (!map.containsKey(nums2[i])) {

stack.push(nums2[i]);

Dry Run

→ Monotonic decreasing stack

[2, 1, 4, 3] → [6, 1, 3, 4, 2, 5, 7]

stack = [1, 3, 4, 2]

res = [5, 3, 5, 4]

what is monotonic decreasing,
whenever it see greater element
it should pop() from top to bottom

503. Next greater element II

$[1, 2, 3, 4, 3] \rightarrow$ circular Array

$[2, 3, 4, 1, 4]$

$\rightarrow$ using monotonic decreasing stack

$[1, 2, 3, 4, 3] [1, 2, 3, 4, 3]$

if $i < N$

res = $[2, 3, 4, -1, 4]$

stack = $[3, 4, 3, 2, 1, 3, 4, 3, 2]$

stack < Integer> stack = new stack<>();

for (i = 2 * N - 1; i >= 0; i--) {

while (!stack.isEmpty() && stack.peek() <= nums[i]) {

Monotonic
decreasing

~~if (i < N) &~~
stack.pop();

}

if (i < N) {

if (stack.isEmpty()) res[i] = -1

else res[i] = stack.peek()

}

stack.push(nums[i % N]);

}